

CSE 232 : Data Structures

LAB 10 : PRIORITY QUEUE

Due Saturday at 5:00 p.m. MST

The tenth programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) `std::priority_queue` class.

Priority Queue

Your list class will be in `priority_queue.h` and be named `custom::priority_queue`. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, move constructor, and range constructor.
- **Access:** top (read-only).
- **Insert:** copy push and move push.
- **Remove:** pop.
- **Status:** size and empty.

A key part of the priority queue implementation is the percolate process. [To see a demo of how this works, please watch this video.](#)

To properly implement the container-move constructor (`test_constructMoveInit_standard()` and company), you will need to implement the `heapify()` method. After calling the container's move assign (using `std::move()`), you will call `heapify()`. The pseudocode is:

```
priority_queue.heapify()  
  for iHeap ← container.size()/2 ... 1  
    percolateDown(iHeap)
```

Three Options

This week, there will be three options.

85 Option : Integer Base Type

This option will require you to implement all the above interfaces from your `vector` class of Week 02. In other words, if you did an 85 vector, then you need to do a 85 vector. This implementation will always use an integer as the base type.

```
custom::vector container;
```

To complete the 85 option, please start with the following files: [232.10.Lab.85.zip](#).

100 Option : Template Base Type

This option will require you to implement all the same interfaces, but supply a template parameter as the base type.

```
template <class T>
```

The underlying data structure will be your `vector` code from Week 02.

```
custom::vector<T> container;
```

To complete the 100 option, please start with the following files: [232.10.Lab.100.zip](#).

115 Option : Container and Comparison

This option will require the class to everything from the 100 option. Additionally, allow the client to specify both the underlying container, and the comparison function.

```
template<class T,
        class Container = custom::vector<T>,
        class Compare = std::less<T>>
```

There are two member variables.

```
Container container;           // underlying container (probably a vector)
Compare compare;              // comparison operator
```

Note that the has code (in particular the `percolateDown()` function) will need to use the `Compare` template variable rather than the less-than operator (`<`).

Hint:: Percolate swaps two values when items are moved in the heap. We use the `swap()` function to accomplish this. Note that we really want to use the template's version of `swap()` if it defined one, but use `std::swap()` if the template did not define one. If you specify `std::swap()` in `percolateDown()`, then it will not use the Spy's version of `swap()` because it's name is not `std::swap()`. If you specify `swap()` in `percolateDown()`, then it will use the Spy's version when the template parameter is a Spy, but will fail to compile for integer priority queues. You can get around this with the following statement at the head of your `percolateDown()` function:

```
using std::swap;
```

To complete the 115 option, please start with the following files: [232.10.Lab.115.zip](#).

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `priority_queue.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

Exceptional	Good	Acceptable	Developing	Missing
100%	90%	70%	50%	0%

Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or was not submitted.
Template Base Type 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Container and Comparison 15%	115 solution was attempted				The 85 or the 100 solution was attempted